

## Examen de la session principale

Module : Algorithmique et Structures de Données

Date : 8/1/2024

Classe : III1

Durée : 2H

Enseignants : D. DHOUIB, I. AMDOUNI, M. BEN SASSI, R. CHEBIL

Documents : N.A

*Il sera tenu compte de la clarté des réponses*

### Exercice 1 (11 points)

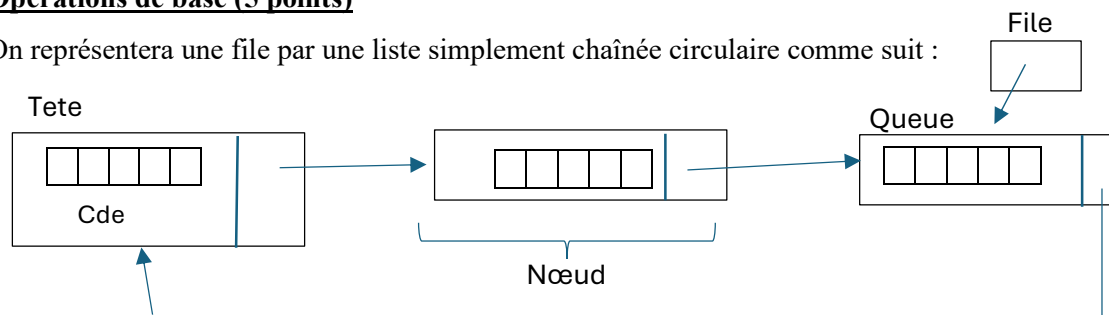
**Dans cet exercice, SEULES les opérations de base sur les PILES peuvent être utilisées sans être développées.**

On s'intéresse dans cet exercice à l'ensemble des commandes effectuées sur le site d'une entreprise de vente en ligne. Chaque commande est caractérisée par une référence unique (exp : NR10257), l'adresse de livraison, la distance en kilomètres du lieu de livraison par rapport au siège de l'entreprise, l'état de confirmation de la commande (vrai si elle est confirmée, faux sinon) et le prix.

Comme le traitement de ces commandes est généralement fait par ordre de leur apparition, celles-ci seront placées dans une file (la commande la moins récente en tête de file et la plus récente en queue). Pour ceci, on commencera dans la première partie par le développement d'une bibliothèque File qu'on utilisera dans la deuxième partie.

#### Opérations de base (5 points)

On représentera une file par une liste simplement chaînée circulaire comme suit :



Comme le nœud qui succède la queue est la tête, la file sera représentée par une seule extrémité qui est la queue.

1. Définir les types Cde, Nœud et File permettant de représenter respectivement la commande, le nœud et la file. (1.5 pt)
2. Ecrire la fonction `est_vide(DON F : File) : booléen` qui étant donnée une file retourne vrai si elle est vide et faux sinon. (0.5 pt)
3. Ecrire la procédure `enfiler(DONRES F : File, DON c : Cde)` qui enfile une commande c dans une file F. (1.5 pt)
4. Ecrire la procédure `defiler(DONRES F : File, RES c : Cde)` qui défile la commande c d'une file F non vide. (1.5 pt)

#### Traitement des commandes (6 points)

5. Chaque jour, les commandes placées dans la file sont nettoyées en enlevant les commandes non confirmées. Ecrire une fonction `nettoyer` qui étant donnée la file de commandes, la nettoie selon le critère précédemment décrit et retourne le nombre de commandes retenues. La file de

*résultat doit contenir uniquement les commandes confirmées. L'utilisation d'une structure intermédiaire **est permise**. (1.5 pt)*

6. Les commandes à livrer à une distance supérieure à une valeur  $d$ , sont affectées à une autre société de livraison. Ecrire une procédure `Sous_traiter` qui étant données la file de commandes confirmées, sa longueur (le nombre de commandes dans la file déjà calculé) ainsi que la distance  $d$ , enlève les commandes vérifiant ce critère et les place dans une autre file de commandes. *L'utilisation de structure intermédiaire en plus des deux files **N'EST PAS permise**. (1.5 pt)*

*On s'intéresse dans tout ce qui suit, aux commandes confirmées et livrées par l'entreprise elle-même.*

7. Chaque jour, seules les  $N$  commandes les moins récentes sont retirées de la file pour être livrées. Ces commandes sont placées dans une liste simplement chaînée **par ordre croissant de distance de livraison** par rapport au siège de l'entreprise (la plus proche au début de la liste et la plus loin à la fin de la liste). Ecrire une fonction qui étant donnée la file et le nombre  $N$ , retourne la liste des  $N$  commandes ordonnées selon le principe précédemment décrit. *On ne demande pas que la liste retournée soit circulaire. (2.5 pts)*
8. Au cours de sa journée de livraison, le livreur utilise une application qui lui permet de retirer les commandes dans leur ordre d'apparition dans la liste précédemment obtenue. Si la commande n'est pas livrée (en général à cause de l'absence du demandeur), il la place en attente dans une autre structure pour essayer de la livrer à son retour au siège de l'entreprise. Si la commande n'est pas livrée au retour, elle est annulée ; on n'a plus besoin de la sauvegarder. Proposer une structure de données adéquate pour que le livreur trouve les commandes en attente dans l'ordre de distance adéquat lors de son retour. Justifier votre proposition. (0.5 pt)

## **Exercice 2 (9 points)**

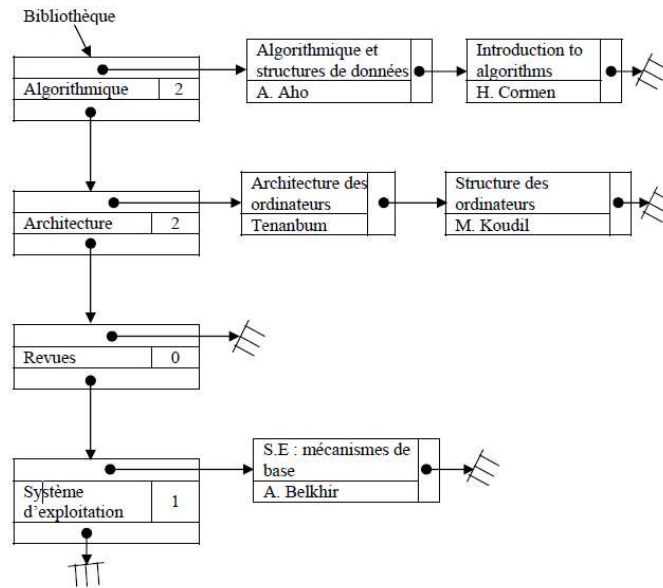
### **Partie 1**

**Dans cet exercice, SEULES les opérations de base sur les PILES et les FILES vues dans le cours peuvent être utilisées sans être développées.**

**On se permet de comparer deux chaînes par l'opérateur =**

Un étudiant en informatique souhaite représenter sa bibliothèque personnelle en utilisant une structure dynamique. La structure proposée est représentée par la figure de la page suivante.

La liste verticale contient les catégories des livres avec le nombre de livres dans chacune, tandis que les listes horizontales contiennent les titres des livres avec leurs auteurs dans chaque catégorie.



1. Donner les structures de données nécessaires à l'implémentation de cette bibliothèque. (1 pt)
2. Ecrire la procédure d'insertion d'un nouveau livre appartenant à une catégorie de nom donné (l'ajout se fait au début de la liste). (1.5 pt)
3. Ecrire la procédure qui permet de supprimer une catégorie donnée avec tous ses livres. (2 pts)

## **Partie 2**

L'étudiant propose d'organiser sa bibliothèque en utilisant un arbre binaire dont chaque nœud est représenté par le titre du livre, son auteur et sa catégorie.

On vous demande de :

4. Donner les structures de données nécessaires à la représentation de sa bibliothèque. (0.5 pt)
5. Ecrire une fonction récursive qui permet de compter et de retourner le nombre de livres de la bibliothèque. (1.5 pt)
6. Ecrire une procédure qui étant donné l'arbre, enregistre son contenu dans un fichier dont le nom est saisi par l'utilisateur. **Utiliser un parcours en largeur.** (2.5 pt)

*Bon Travail*